SOURCE-LEVEL GUIDE • OPENSANDBOX-GROUP/OPENSANDBOX

OpenSandbox 源码导读

OpenSandbox 让 AI 应用在自己的基础设施上运行隔离沙箱。这份导读讲清控制面、数据面和网络面怎么分工，一次 Sandbox.create 和 commands.run 怎么走完，以及 Fast Sandbox、出站控制和凭据保险库的设计。

opensandbox-group/OpenSandbox @ 0da12cf

怎么读这份导读

这份导读写给想部署、贡献或定制 OpenSandbox 的工程师。读完你应该能说清：控制面和数据面为什么分开；一次创建和一次命令执行分别经过哪些组件；Docker、Kubernetes、Fast Sandbox 三种后端有什么差别；新运行时、新 SDK 能力该改哪里。

全文顺序：

1. **全景**：问题、目标、限制，六个面。
2. **协议**：`specs/` 是公共合约。
3. **控制面**：生命周期服务器。
4. **运行时后端**：Docker、Kubernetes、Fast Sandbox。
5. **数据面**：execd。
6. **网络面**：ingress、egress、凭据保险库。
7. **Kubernetes 控制器**：CRD 和资源池。
8. **SDK 与工具**。
9. **端到端**：从 `Sandbox.create` 到一条命令。
10. **许可证、贡献与定制、源码索引**。

> **要点 · 固定版本**
>
> `opensandbox-group/OpenSandbox` @ `0da12cf`（2026-11-06）。我没有部署或运行过，内容来自阅读代码和仓库文档。性能数字都来自项目自己的文档，我没有复测。

全景：问题、目标、限制

它解决什么

AI 应用要执行模型生成的代码、跑命令、操作浏览器。这些事不能在应用进程里做。

环节	问题	解决办法
隔离	模型生成的代码不可信	容器、gVisor/Kata/Firecracker，加网络控制
环境可移植	本地用 Docker，生产用 K8s，代码不想改两遍	统一的生命周期 API
启动延迟	每轮 agent 都等容器启动太慢	资源池、Fast Sandbox 模板
凭据泄露	agent 要调外部 API，但不能拿到密钥	出站代理注入凭据
评估规模	RL 和 agent 评估要大批量沙箱	批量创建、资源池

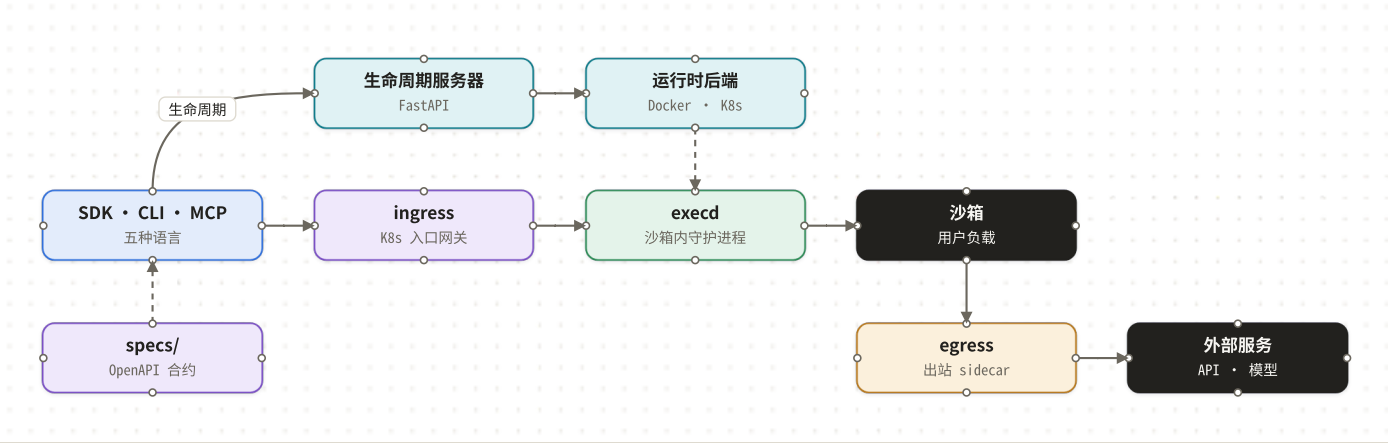
要求 / 目标 / 限制

项目	内容
核心目标	通用沙箱平台，运行在用户自己的基础设施上
功能要求	生命周期、命令、文件、代码、PTY、端点、快照、暂停恢复、出站策略
规模压力	README 给出 Firecracker 创建 64ms P50（串行）、125.4ms P99（10 并发）（`Features`）
外部限制	多语言 SDK 要一致；Docker 和 K8s 能力不同；Fast Sandbox 平台在别的仓库
关键手段	OpenAPI 合约；统一 `SandboxService`；数据面直连；fail-closed 出站
许可证	Apache-2.0，LF Projects 技术章程，AAIF 项目

六个面

`架构总览` 把项目分成六个面，这是读仓库最好的地图：

控制面和数据面分开走



三条路径：生命周期走服务器；执行和服务访问**绕过服务器**直连沙箱；出站经 egress 过滤。这个分离来自 架构文档 。Docker 模式没有 ingress，客户端直连映射端口。

面	目录
客户端	`sdks/` 、 `cli/`
协议	`specs/`
控制面	`server/`
运行时	`services/` 、 `kubernetes/`
数据面	`components/execd`
网络与安全	`ingress` 、 `egress`

协议：specs 是公共合约

`specs/` 下有四份 OpenAPI：`生命周期`、`execd`、`egress`、`诊断`。

根 `AGENTS.md` 的规则是：把 `specs/*` 当公共合约；用户可见的行为变化时，spec、实现、SDK、文档、示例要一起对齐，**先改文档**。扩展优先做向后兼容的增量。

这种设计的好处是可替换：任何实现了 execd API 的镜像都能配合 SDK 工作。`execd 文档` 称它是“参考”守护进程。

> **类比 · 插座标准和电器**
>
> - **插座标准** = `specs/` 的 OpenAPI。
> - **电器** = 五种语言的 SDK。
> - **电网** = Docker、K8s、Fast Sandbox 后端。
>
> 电器不知道电从哪来，换电网不用换电器。所以有一条硬规则：不能只改生成物来修问题，要改标准本身。

控制面：生命周期服务器

职责

一个 FastAPI 应用，是所有 SDK 共用的认证入口，只管生命周期。入口在 `main.py`，路由在 `lifecycle.py`。

两条分层规则

`server.md` 写得很明确：

1. **API 层不直接接触运行时。** 一切经过服务接口。
2. **沙箱不是服务器数据库里的行。** Docker 容器、K8s 资源或 Fast Sandbox CR 才是事实来源。服务器只持久化自己管理的记录，比如快照元数据和模板目录。

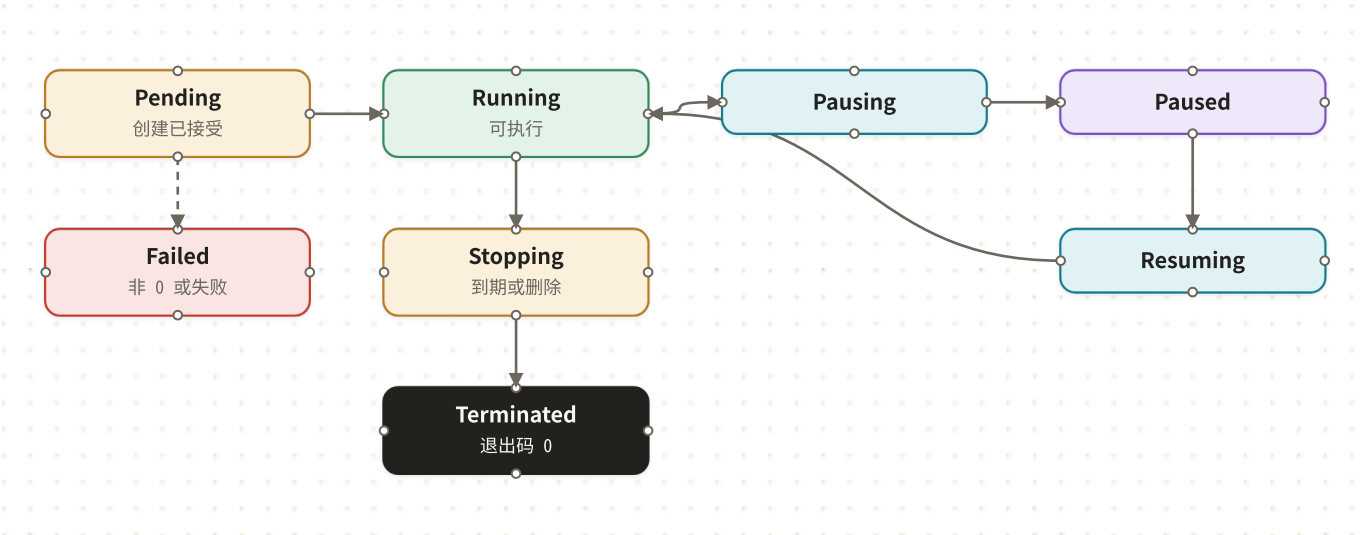
第二条很关键：服务器本身几乎无状态，容器的 label 就是元数据。比如 `createdAt` 存在 `opensandbox.io/created-at` 这个 label 里。

层	目录	职责
API	`api/`	薄路由
中间件	`middleware/`	认证、请求 ID、指标
服务	`services/`	状态机和运行时集成
仓储	`repositories/`	快照元数据，SQLite 或 PostgreSQL
租户	`tenants/`	多租户身份，只用于 K8s
集成	`integrations/`	空闲暂停、访问续期、OTel

生命周期是异步的

创建立即返回 `Pending`，客户端轮询或用 SDK 的就绪辅助函数。状态有 `Pending`、`Running`、`Pausing`、`Paused`、`Resuming`、`Stopping`、`Terminated`、`Failed`（`Asynchronous lifecycle`）。

沙箱状态



每个沙箱有绝对过期时间，通过 `renew` 延长。K8s 的删除是异步的，`DELETE` 返回 204 后要轮询到它消失。

启动时就拒绝不安全的配置

服务器遵循 fail-fast：非交互环境没有 API key 是致命错误，除非显式确认不安全（`startup_guard.py`）。

服务接口

`SandboxService` 是抽象基类，`create_sandbox_service` 按配置选实现。

运行时后端

后端	实现	选用	特点
Docker	`DockerSandboxService`	`runtime.type = "docker"`	本地和单机
Kubernetes	`KubernetesSandboxService`	`"kubernetes"`	BatchSandbox 或 agent-sandbox
Fast Sandbox	`FastSandboxService`	同样是 `kubernetes`，请求带 `templateId`	模板化 Firecracker

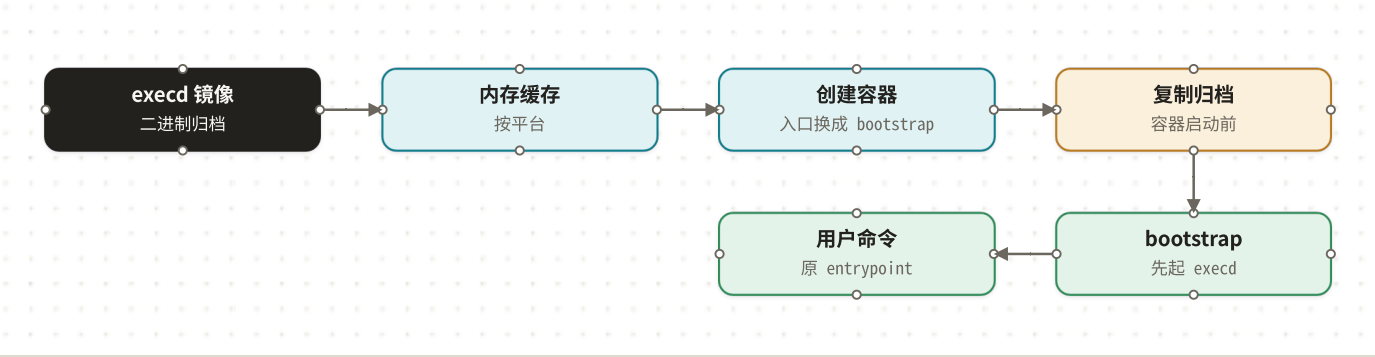
Fast Sandbox **不是**单独的 `runtime.type`。`CompositeSandboxService` 在 K8s 下按请求路由：带 `templateId` 的走 Fast Sandbox，ID 前缀 `fsb-` 决定后续操作归谁（`How it fits`）。不用新增公共 API，SDK 不用改。

Docker 怎么把 execd 放进任意镜像

用户给的是任意镜像，比如 `alpine`，里面没有 execd。Docker 后端的做法是：

1. 从 execd 镜像取出二进制归档，按平台缓存在内存（`runtime.py`）。
2. 创建容器，入口改成 bootstrap 脚本，原命令作为它的参数（`container_ops.py`）。
3. 把归档复制进容器，再启动（`_prepare_sandbox_runtime`）。

把 execd 注入任意镜像



这样 `alpine` 这种没有准备过的镜像也能用同一套 execd API。`execd_run_as_init` 配置可以让 execd 作为 init 运行（`代码`）。

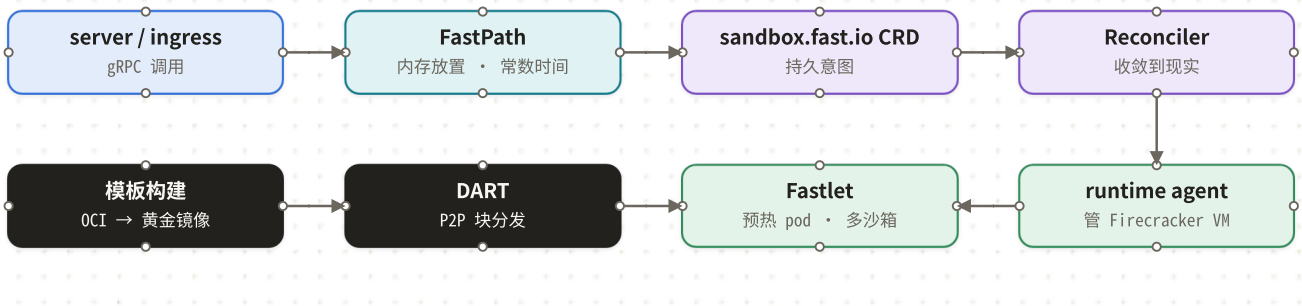
Kubernetes 用 init container 做同样的事（`batchsandbox_provider.py`）。Fast Sandbox 把 execd 烤进模板。

隔离档位

`SecureRuntimeConfig` 支持 `gvisor`、`kata`、`firecracker`。Firecracker 通过 `kata-fc` 只能用于 Kubernetes。默认是普通容器，也就是和宿主共享内核。**处理不可信代码时，一定要配置 secure runtime。**

Fast Sandbox

Fast Sandbox 的关键组件



组件表见 `Core components`。**FastPath、Fastlet 和 CRD 属于外部的 fast-sandbox 平台**，这个仓库只有适配器。

快的关键是：工作负载在请求之前就固定了。模板把镜像预构建成黄金镜像，创建只是表达意图，不是配置。Fastlet 预置好网络槽位，放置在内存中完成，不走 K8s 调度器。

数据面：execd

职责

`execd` 是沙箱内的 Go 守护进程，提供命令、代码、PTY、文件、指标。SDK 的命令和文件操作都用这个 API。

区域	端点	说明
健康	`GET /ping`、`/ready`	不需要 token
命令	`/command`	前台流式，或后台轮询
会话	`/session`	有状态 shell
代码	`/code`、`/code/contexts`	基于 Jupyter
终端	`/pty`	WebSocket
文件	`/files`、`/directories`	完整文件系统操作
隔离会话	`/v1/isolated/*`	私有命名空间
指标	`/metrics`	快照和流
代理	`/proxy/{port}`	通过 execd 访问任意端口

表格整理自 `execd.md`。配置了 token 时，请求要带 `X-EXECD-ACCESS-TOKEN`。

代码入口：路由到 `command.go` 的 `RunCommand`，执行在 `runtime/`。

`/proxy/{port}` 解决了一个单机问题：一个映射的宿主端口就能到达沙箱内所有端口（`Single-Host Network`）。

> **注意 · 容易误解的地方**
>
> - **execd 和你的代码在同一个信任域**，隔离在沙箱这一层，不在 execd 这一层。
> - **执行不经过服务器**。所以服务器日志里看不到命令，审计要在别处做。
> - `renew` **不会自动发生**。默认过期就没了，可以用 renew-on-access 集成。

网络面

入站

Kubernetes 上，`ingress` 是所有沙箱共用的反向代理，不需要逐沙箱配置。路由键是“沙箱 ID + 端口”：

模式	写法
头部（默认）	`OpenSandbox-Ingress-To: <id>-<port>`，或主机名 `id-8080.example.com`
URI	`/<id>/<port>/<path>`
签名令牌	带过期和 MAC，不用查表就能验证

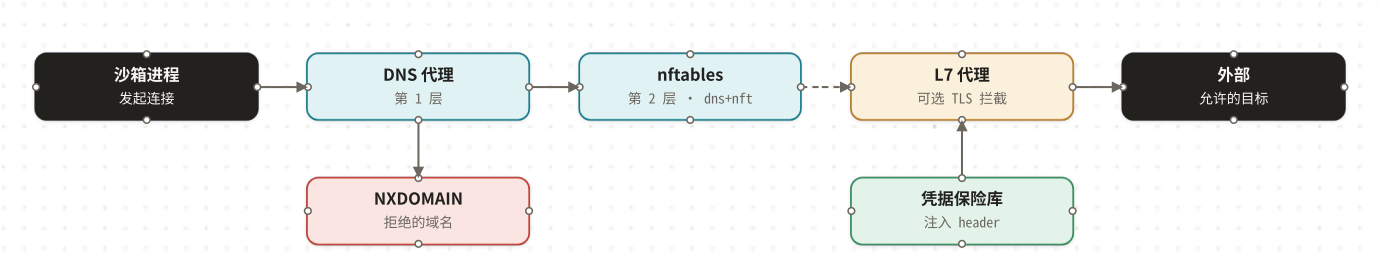
错误码是合约的一部分：`404` 是沙箱不存在，`503` 是存在但还没就绪（`Error semantics`）。客户端可以据此分支。

客户端不猜地址，`GET /sandboxes/{id}/endpoints/{port}` 返回可达地址（`get_sandbox_endpoint`）。

出站

`egress` 作为 sidecar 共享沙箱网络命名空间，所有出站包都经过它（`egress.md`）。

出站的两层



规则优先级：平台 `deny.always` → 平台 `allow.always` → 用户策略，第一条匹配生效。启动失败就退出，不会以空操作运行。

模式	强度
`dns`（默认）	只过滤解析。直接连 IP 可以绕过
`dns+nft`	包级默认拒绝，地址由 DNS 结果带租约喂入

默认的 DNS 模式挡不住直连 IP，这一点要清楚。需要强保证就用 `dns+nft`。端口规则目前只支持 TCP（`egress.md`）。

凭据保险库

`Credential Vault` 让 agent 调外部服务而看不到真实凭据：真实凭据写进 egress sidecar，出站时注入。它要求 `dns+nft` 模式，因为 DNS 模式下直连 IP 能绕过策略。

类比 · 公司前台的门卡

- **员工** = 沙箱里的 agent，不持有大楼门卡。
- **前台** = egress sidecar，门卡放在前台抽屉。
- **出门办事** = 出站请求：经过前台时，前台代为刷卡。
- **只能去名单上的地方** = 出站策略。

员工可以把工作证（假 token）给任何人看，但真门卡从未到过他手上。

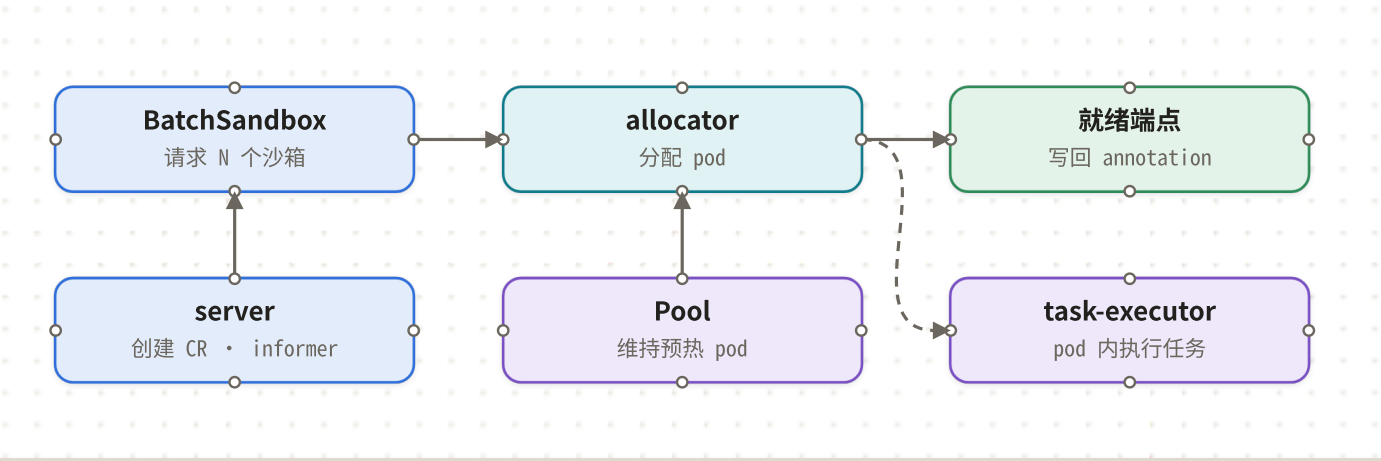
Kubernetes 控制器

`kubernetes/` 是一个独立的 Go operator，有三个 CRD：

CRD	作用	定义
`BatchSandbox`	一批沙箱	`batchsandbox_types.go`
`Pool`	预热资源池	`pool_types.go`
`SandboxSnapshot`	暂停和快照	`sandboxsnapshot_types.go`

控制器在 `internal/controller`：`batchsandbox_controller.go`、`pool_controller.go`、`allocator.go` 负责把池里的 pod 分给 BatchSandbox。

资源池分配



server 用 informer 读状态（`informer.py`）。CRD 是公共接口，改 `apis/` 要跑 `make manifests` 和 `make generate`（`kubernetes/AGENTS.md`）。

另一个可选后端是 `kubernetes-sigs/agent-sandbox`，实现在 `agent_sandbox_provider.py`。

SDK 与工具

工具	位置
Python、JS/TS、Kotlin/Java、C#、Go SDK	`sdks/sandbox`
Code Interpreter SDK	`sdks/code-interpreter`
MCP	`sdks/mcp`
`osb` CLI	`cli/`
控制台	`console/`
Helm	`manifests/charts`

生成的 OpenAPI 客户端和手写适配层并存：生成代码处理普通请求，手写层处理流式、传输、错误（`Client surface`）。**永远不要只改生成物**。

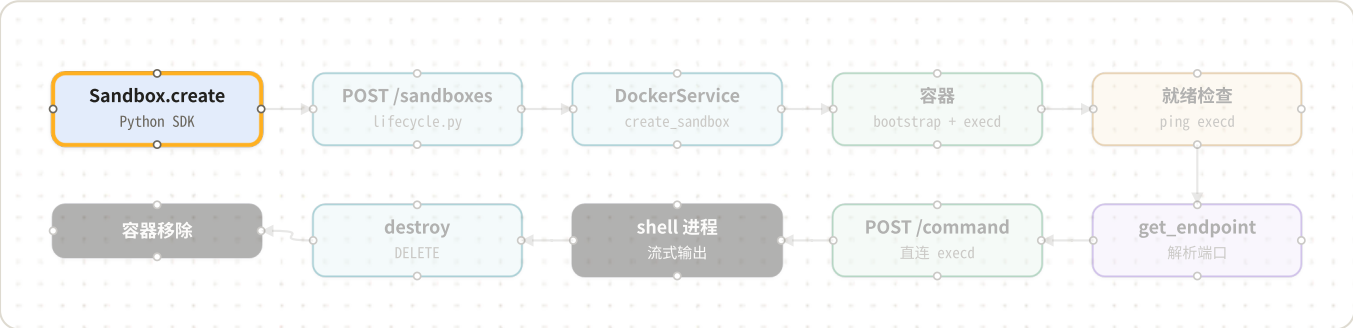
代码解释器镜像已经搬到 `opensandbox-group/sandbox-images`，不在这里。

端到端：从 create 到一条命令

```
sandbox = await Sandbox.create("alpine")
execution = await sandbox.commands.run("echo 'Hello OpenSandbox!'")
await sandbox.destroy()
```

简化自 `README`。

▶ Docker 模式下的创建和执行 1 / 10



1. SDK 发起

`Sandbox.create` 通过生成的客户端调用生命周期 API，默认 `ready_timeout` 是 30 秒。

2. 路由

`create_sandbox` 校验扩展字段和网络策略，然后交给 `sandbox_service`。路由本身很薄。

3. 服务

`create_sandbox` 准备资源限制和 ID，调用 `_provision_sandbox`。

4. 建容器

`_create_and_start_container` 替换入口，复制 execd 归档，启动。容器的 label 记录元数据。

5. 等就绪

SDK 解析端点并做健康检查，直到 execd 响应或超时。

6. 取端点

`get_endpoint` 通过生命周期 API 得到可达地址。

7. 执行

`commands.run` 直接请求 execd 的 `/command`，这一步**不经过服务器**。

8. 流式

execd 启动 shell，把输出流式返回。

9. 销毁

`destroy` 调 `DELETE /sandboxes/{id}`。

10. 清理

容器移除。没有显式删除时，到期也会回收。

K8s 模式的差别：创建变成 CR，init container 放 execd，端点经 ingress。

开源许可证

部分	许可证	证据
全部	Apache-2.0	`LICENSE`
项目治理	LF Projects 技术章程	`CHARTER.md`、`GOVERNANCE.md`
依赖	按各生态的清单	我没有逐个审计

场景	允许吗	要做什么
自用	允许	无
提 PR	允许	**需要 DCO**：`git commit -s`（`CONTRIBUTING`）
公开 fork	允许	保留 LICENSE，商标按章程
发安装包	允许	保留声明
对外提供服务	允许	Apache-2.0 无网络条款

历史：GitHub API 显示 `LICENSE` 有两条记录：2025-11-13 和 2025-12-17，第一条标题就是添加 Apache 2.0。包名里还保留 `alibaba`（`README`），Go 模块路径也是，说明项目最初来自阿里巴巴，这是根据命名的推断。

> **不是法律意见**
>
> 这是工程师读代码时的理解。

贡献与定制

开发环境

```
cd server
uv sync --all-groups
cp opensandbox_server/examples/example.config.toml ~/.sandbox.toml
uv run python -m opensandbox_server.main
uv run pytest tests/test_docker_service.py
```

来自 `Quick Setup` 和 `server/AGENTS.md`。

项目规则

- 根 AGENTS.md 是路由器，先读最近目录的 `AGENTS.md`。
- 公共 API、SDK、CLI、生命周期语义、运行时隔离、网络的改动，走 `OSEP` 提案（`Governance`）。
- 路由保持薄，逻辑放服务层。
- Docker 能跑不代表 K8s 能跑。
- 不要只改生成物。
- 破坏性变更要先问。
- 提交带 DCO。

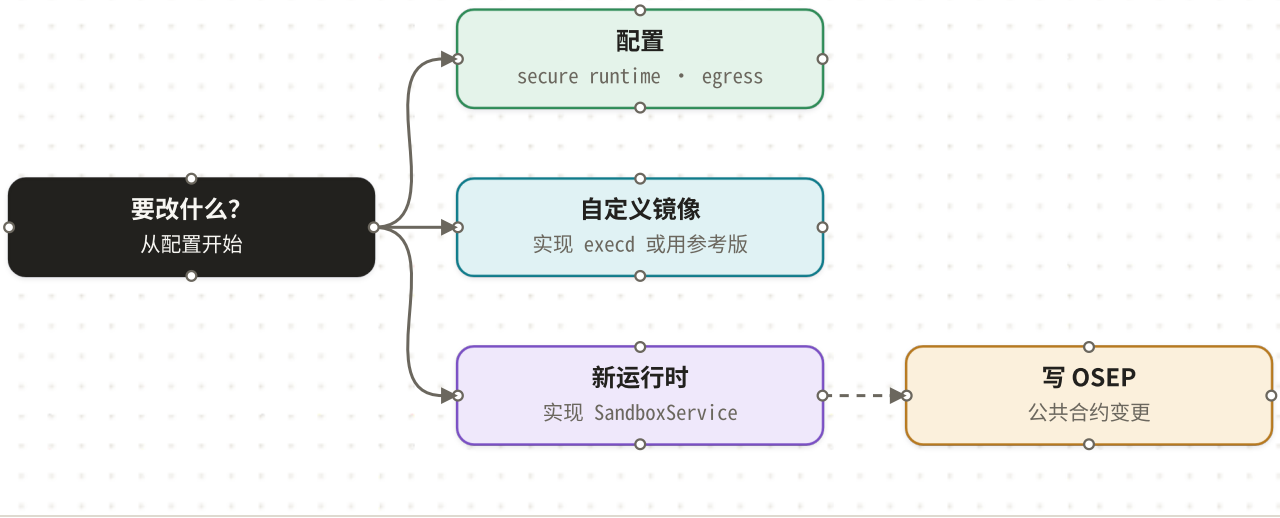
适合入手的方向

- 文档和示例：`docs/examples`。
- SDK 能力对齐：五种语言容易有差异。
- 诊断 API 和错误信息。
- `ROADMAP` 上的条目。

需要避开：没有 OSEP 就改公共合约。

定制路线

想让它适配我的环境



配置示例在 `example.config.toml`。

仓库里没有的东西

- **fast-sandbox 平台**：FastPath、Fastlet、CRD 在 fast-sandbox。
- **官方沙箱镜像**：在 `sandbox-images`。
- **DART**：外部的块分发项目。
- **Firecracker、gVisor、Kata**：要自己准备。

附录：源码索引

主题	链接
路由表	`AGENTS.md`
架构总览	`index.md`
合约	`specs/README.md`
服务器入口	`main.py`
配置	`config.py`
生命周期 API	`lifecycle.py`
Docker	`docker_service.py`
K8s	`kubernetes_service.py`
execd	`main.go`
egress	`main.go`
ingress	`main.go`
控制器	`batchsandbox_controller.go`
Python SDK	`sandbox.py`
Fast Sandbox	`index.md`
治理	`GOVERNANCE.md`、`CHARTER.md`